

PNG 图像格式、解码算法与工程实现

从文件格式、Chunk、zlib、DEFLATE，到 Scanline 重建、Adam7、颜色处理与硬件架构

版本 0.9 · 2026-08-19

00. 前言

从文件格式、Chunk、zlib、DEFLATE，到 Scanline 重建、Adam7、颜色处理与硬件架构

版本：0.9

日期：2026-08-19

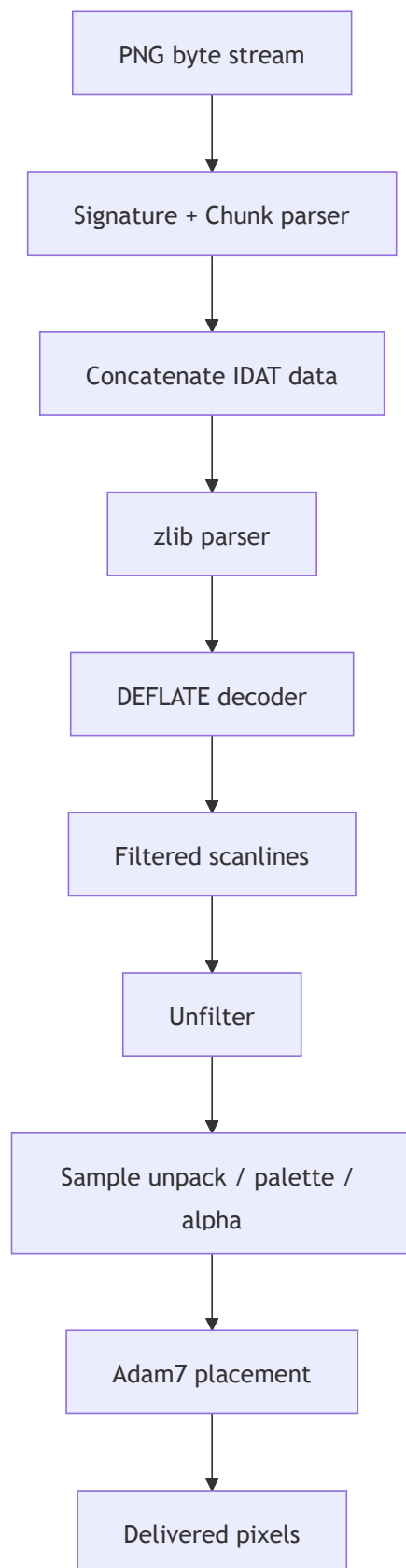
适用对象：PNG 初学者、软件解码器开发者、FPGA/ASIC/RTL 设计工程师

前言

PNG (Portable Network Graphics) 不是“把像素直接交给 DEFLATE 压缩”这么简单。它是一套分层的数据格式：

1. 最外层是 PNG Signature 和一系列 Chunk；
2. IDAT Chunk 的 Data 字段拼接成一个 zlib 数据流；
3. zlib 内部包裹一个 DEFLATE 位流，并在尾部提供 Adler-32；
4. DEFLATE 解压得到的不是最终像素，而是带有行过滤器类型的 Filtered Scanline；
5. Decoder 必须执行反滤波、位深解包、调色板与透明度处理；若使用 Adam7，还要把七个 Pass 的像素放回原图位置。

本书以解码器为主线，同时补充编码器和硬件实现。核心数据路径如下。



规范依据

本书主要依据以下标准：

- W3C, Portable Network Graphics (PNG) Specification, Third Edition, W3C Recommendation, 24 June 2025;
- ISO/IEC 15948:2004, 与 PNG Second Edition 对应;
- RFC 1950, ZLIB Compressed Data Format Specification version 3.3;
- RFC 1951, DEFLATE Compressed Data Format Specification version 1.3;
- W3C PNG Fourth Edition Editor's Draft, 用于了解后续演进, 不把草案内容误写成既有实现的强制要求。

规范链接见文末参考资料。

阅读约定

- byte 固定指 8 bit;
- PNG 多字节整数采用网络字节序, 即最高有效字节在前;
- 所有 Scanline Filter 运算都按 byte 进行, 并对 256 取模;
- 伪代码强调算法含义, 不绑定具体语言;
- 本文中的“必须”“不得”表示格式或一致性约束; “建议”表示工程选择。

目录

- 第一篇 建立整体认识
- 第二篇 PNG 文件格式与 Chunk
- 第三篇 像素与 Scanline
- 第四篇 完整 Decoder 数据流
- 第五篇 zlib
- 第六篇 DEFLATE 解码
- 第七篇 Scanline Filter 与重建
- 第八篇 Adam7
- 第九篇 颜色、Alpha 与输出

- 第十篇 编码器
 - 第十一篇 硬件架构
 - 第十二篇 错误、安全与验证
 - 第十三篇 贯通实例
 - 附录 A~E 公式、误解、APNG、版本关系与参考资料
-

第一篇 建立整体认识

1. PNG 是什么

PNG 是一种无损、可扩展、可流式解析的栅格图像格式。它支持：

- 灰度、真彩色和索引色；
- 1、2、4、8、16 bit Sample Depth；
- 可选 Alpha；
- Gamma、色度、ICC、HDR 标识等颜色信息；
- 文本、Exif、物理尺寸、时间等元数据；
- Adam7 渐进显示；
- CRC 和 Adler-32 两层完整性检查；
- 在较新规范中定义的 APNG 帧动画。

“无损”意味着：忽略显示设备和颜色管理造成的视觉转换，Decoder 能逐 Sample 恢复编码器提交给 PNG 编码过程的数据。

PNG 的压缩效果来自两层协作：

1. Scanline Filter 把空间相关性转化为大量小残差；
2. DEFLATE 用 LZ77 查找重复串，再用 Huffman Code 表示 Literal、Length 和 Distance。

Filter 自身不减少字节数。每行反而多出一个 Filter Type Byte；真正减少数据量的是后面的 DEFLATE。

2. 编码与解码的对称关系

编码路径：

text

Reference pixels

- > optional Adam7 pass extraction
- > scanline serialization
- > per-scanline filtering
- > concatenate all filtered scanlines
- > zlib/DEFLATE compression
- > split compressed bytes across IDAT chunks
- > construct PNG chunks and CRCs

解码路径：

text

PNG bytes

- > parse chunks and verify CRC
- > concatenate IDAT data fields
- > parse zlib header
- > decode DEFLATE
- > verify Adler-32
- > split output into pass scanlines
- > reverse the scanline filters
- > unpack samples and apply palette/transparency
- > place Adam7 samples into final coordinates

需要牢牢记住三个边界：

- IDAT Chunk 边界只是 PNG 封装边界；
- DEFLATE Block 边界是压缩算法边界；
- Scanline 边界是图像预测边界。

三者通常互不对齐。Decoder 不能假设 “一个 IDAT 对应一个 DEFLATE Block” 或 “一个 Block 对应一行” 。

3. 六层数据模型

层次	输入	输出	主要职责
PNG 文件层	文件字节	Chunk	Signature、顺序、长度、CRC
IDAT 聚合层	一个或多个 IDAT	连续 zlib 字节流	去掉 Chunk 外壳并按顺序拼接 Data
zlib 层	zlib 字节流	DEFLATE 位流与校验结果	CMF/FLG、Adler-32
DEFLATE 层	压缩位流	Filtered Scanline 字节	Block、Huffman、LZ77
Filter 层	Filter Type + Filtered Bytes	原始 Scanline Bytes	None/Sub/Up/Average/Paeth
像素层	Scanline Bytes	像素或 Sample	位深解包、PLTE、tRNS、Adam7

这种分层也应直接反映在软件模块或 RTL 模块划分中。

第二篇 PNG 文件格式与 Chunk

4. PNG Signature

每个 PNG 数据流以前8字节开始：

89 50 4E 47 0D 0A 1A 0A

text

其中 50 4E 47 是 ASCII 的 “PNG” 。其余字节帮助检测：

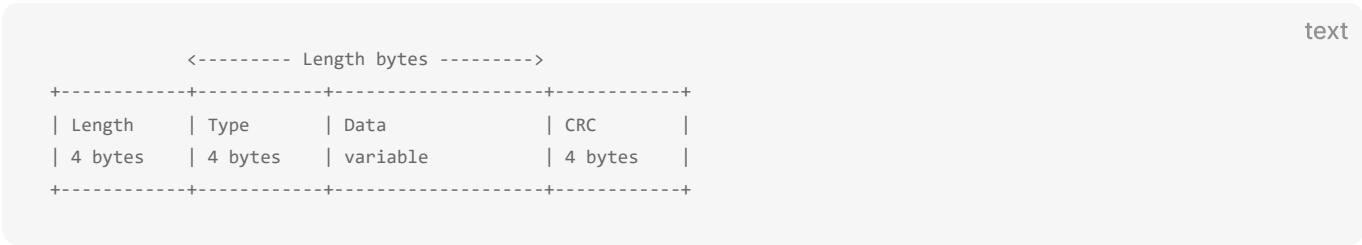
- 文本模式与二进制模式混淆；
- CR/LF 换行转换；
- 文件截断；
- 某些旧式传输链路对控制字符的破坏。

Decoder 应在分配大块图像内存之前检查完整 Signature。

5. Chunk 通用结构

每个 Chunk 均为：

字段	大小	含义
Length	4 byte	Data 字段长度，不含 Type 和 CRC
Type	4 byte	四个 ASCII 字母
Data	Length byte	Chunk-specific payload
CRC	4 byte	对 Type 与 Data 计算的 CRC-32



Length、宽高及其他 PNG 整数均按大端字节序读取。实现时必须先做边界与溢出检查，再计算：

text

chunk_total = 12 + length

不要在未验证 length 的情况下直接执行内存分配或指针加法。

6. Chunk Type 的四个属性位

Chunk Type 的四个字符必须是英文字母。每个字符 ASCII 值的 bit 5，即大小写差异，具有独立含义。

字符位置	大写, bit 5=0	小写, bit 5=1
第1字符	Critical	Ancillary
第2字符	Public	Private
第3字符	Reserved, 合法类型必须大写	保留, 当前不合法
第4字符	Unsafe-to-copy	Safe-to-copy

例如 IDAT 的首字符 I 为大写，所以它是 Critical Chunk。tEXt 首字符小写，所以它是 Ancillary Chunk。

未知 Critical Chunk 意味着 Decoder 不知道如何正确解释图像，必须报错。未知 Ancillary Chunk 通常可以跳过，但编辑器是否可在修改图像后原样保留，还要看第四字符的 safe-to-copy 属性。

7. IHDR：图像的总合同

IHDR 必须是 Signature 后的第一个 Chunk，只出现一次，Data 长度固定13字节。

偏移	字段	大小
0	Width	4
4	Height	4
8	Bit Depth	1
9	Color Type	1
10	Compression Method	1
11	Filter Method	1
12	Interlace Method	1

Width 和 Height 必须非零，且不能超过 PNG 四字节无符号整数允许的范围。

7.1 Color Type 与通道数

Color Type	含义	通道顺序	通道数
0	Grayscale	G	1
2	Truecolor	R, G, B	3
3	Indexed-color	Palette Index	1
4	Grayscale with Alpha	G, A	2
6	Truecolor with Alpha	R, G, B, A	4

Color Type 的 bit 含义可以帮助记忆，但实现必须只接受规范定义的 0、2、3、4、6。

7.2 合法 Bit Depth

Color Type	1	2	4	8	16
0	✓	✓	✓	✓	✓
2				✓	✓
3	✓	✓	✓	✓	
4				✓	✓
6				✓	✓

7.3 三个 Method 字段

- Compression Method：当前只定义0；
- Filter Method：当前只定义0；
- Interlace Method：0 表示无交错，1 表示 Adam7。

PNG Compression Method 0 规定使用 zlib/DEFLATE。这里的“0”是 PNG 的方法编号，不是 zlib Header 中的 CM 值。

8. PLTE：调色板

PLTE Data 由若干个三字节条目组成：

R0 G0 B0 | R1 G1 B1 | ...

text

约束包括：

- 长度必须是3的倍数；

- 条目数为1至256;
- Color Type 3 必须有 PLTE;
- Color Type 0 和4 不得出现 PLTE;
- Color Type 2 和6 可以提供建议调色板;
- PLTE 必须位于第一个 IDAT 之前;
- Color Type 3 的条目数不得超过位深能表示的索引数量;
- 解码得到的索引超出实际 PLTE 条目数属于错误。

PLTE 不进入 DEFLATE。它是独立 Chunk。

9. IDAT：图像压缩数据

IDAT 的 Data 字段包含 zlib 数据流的一部分。PNG 可以有一个或多个 IDAT，Decoder 必须把所有连续 IDAT 的 Data 按文件顺序逻辑拼接：

```
IDAT_0.Data || IDAT_1.Data || ... || IDAT_n.Data
```

text

拼接结果是一个 zlib datastream：

```
CMF | FLG | DEFLATE blocks ... | ADLER32
```

text

关键结论：

1. IDAT 的 Length、Type 和 CRC 不属于 zlib;
2. IDAT Data 开头不一定是新的 zlib Header，只有整个拼接流开头才是;
3. Chunk 可以在任意压缩字节位置切分;
4. Chunk 边界甚至可能落在一个 Huffman Code 所在字节附近;
5. Decoder 可以流式地把每个 IDAT Data 送入同一个持久化 zlib 状态机，不必先复制到一块大缓冲。

严格说，“去掉 IDAT Header 后 body 就是 zlib 标准格式”只对所有 IDAT Data 的有序拼接成立，不对每个 IDAT 单独成立。

10. IEND 与 Critical Chunk 顺序

IEND:

- Data 长度必须为0;
- 只出现一次;
- 必须是最后一个 PNG Chunk。

静态 PNG 的关键顺序可概括为:

```
IHDR  
[PLTE]  
IDAT ...  
IEND
```

text

所有 IDAT 必须连续出现。如果已经离开 IDAT 序列又遇到 IDAT，应视为顺序错误。

11. Ancillary Chunk 速览

类别	Chunk	作用
透明度	tRNS	透明色或 Palette Alpha
颜色	cHRM	白点和原色色度
颜色	gAMA	图像 Gamma
颜色	iCCP	内嵌 ICC Profile
颜色	sRGB	标准 sRGB 意图
颜色	cICP	Color Primaries、Transfer、Matrix、Range
HDR	mDCv	Mastering Display Color Volume
HDR	cLLi	Content Light Level
精度	sBIT	原始有效位数
文本	tEXt	Latin-1 文本
文本	zTXt	压缩 Latin-1 文本
文本	iTXt	UTF-8 国际文本
显示	bKGD	建议背景色
调色板	hIST	Palette 频率
物理	pHYs	像素密度或宽高比
调色板	sPLT	建议调色板
元数据	eXIf	Exif Profile
时间	tIME	最后修改时间
动画	acTL/fcTL/fdAT	APNG 控制、帧控制和帧数据

基础图像 Decoder 可以忽略多数不理解的 Ancillary Chunk，但不能忽略影响核心像素解释的 IHDR、PLTE、IDAT、IEND，也不能在遇到未知 Critical Chunk 后继续声称解码正确。

11.1 tRNS

tRNS 的结构依赖 Color Type:

- Type 0: 一个灰度 Sample 值;
- Type 2: R、G、B 三个 Sample 值;
- Type 3: 按 Palette Index 排列的 Alpha byte, 未提供的尾部条目默认为255;
- Type 4、6 已有 Alpha Channel, 不允许 tRNS。

对 Type 0/2, 透明判断必须在原始 Sample 精度下做完全相等比较。

11.2 颜色 Chunk

颜色管理不是恢复像素编码值的必要条件, 却是正确显示颜色的重要条件。工程上至少应区分:

- “Decoder 恢复了 PNG Sample” ;
- “Viewer 把 Sample 转换为显示设备颜色” 。

Alpha 总是线性的, 不能套用颜色通道的 Gamma 曲线。PNG 存储的是非预乘 Alpha (straight/unassociated alpha) ; 如果输出接口要求 premultiplied alpha, 应在颜色空间处理正确后显式转换。

12. CRC-32

每个 Chunk 的 CRC 覆盖:

Type bytes || Data bytes

text

不覆盖 Length。PNG 使用标准 CRC-32 多项式, 反射实现常用 0xEDB88320。典型流程:

text

```
crc = 0xFFFFFFFF
for byte in Type || Data:
    crc = update_crc(crc, byte)
crc = crc XOR 0xFFFFFFFF
```

流式 Parser 可以在读取 Type 和 Data 时同步更新 CRC，无需缓存整个 Chunk。

CRC 的价值是定位 Chunk 级传输错误；zlib 尾部 Adler-32 则覆盖解压后的全部 Filtered Scanline 字节。两者保护的对象和层级不同。

第三篇 像素与 Scanline

13. Pixel、Sample、Channel

Pixel 是空间位置；Channel 是 G、R、B、A 或 Palette Index 等分量；Sample 是某个 Pixel 与某个 Channel 的交点。

例如 8-bit RGBA：

```
Pixel 0: R0 G0 B0 A0  
Pixel 1: R1 G1 B1 A1
```

text

Bits per pixel：

```
bits_per_pixel = channels * bit_depth
```

text

一条非交错 Scanline 的数据字节数：

```
row_bytes = ceil(width * bits_per_pixel / 8)  
           = (width * bits_per_pixel + 7) / 8
```

text

实现必须用足够宽的整数并检查乘法溢出。

14. Sample 打包

14.1 Bit Depth 小于8

1、2、4-bit Sample 从每个 byte 的最高有效位向最低有效位打包。每行独立开始，末尾不足一个 byte 的低位是 Padding，不属于图像 Sample。

例如 2-bit 灰度 Sample：

```
samples: 3, 1, 0, 2
bits:    11 01 00 10
byte:    11010010 = D2
```

text

不能把上一行末尾的剩余 bit 与下一行拼在一起。

14.2 Bit Depth 16

每个 16-bit Sample 按大端顺序：

```
sample = high_byte << 8 | low_byte
```

text

RGBA16 的一个像素占8 byte，顺序为 R高、R低、G高、G低、B高、B低、A高、A低。

15. Filtered Scanline 格式

DEFLATE 解压后，每个非空 Scanline 的格式是：

```
FilterType | filtered_byte[0] | ... | filtered_byte[row_bytes-1]
```

text

因此非交错图像的预期解压长度为：

```
expected = height * (1 + row_bytes)
```

text

Adam7 图像要分别计算七个 Pass，每个非空 Pass 的每行也多一个 Filter Type Byte。

15.1 Filter 算法中的 bpp

Filter 公式中的 bpp 定义为一个完整像素占用的字节数，向上取整：

```
bpp = max(1, ceil(bits_per_pixel / 8))
```

text

例如：

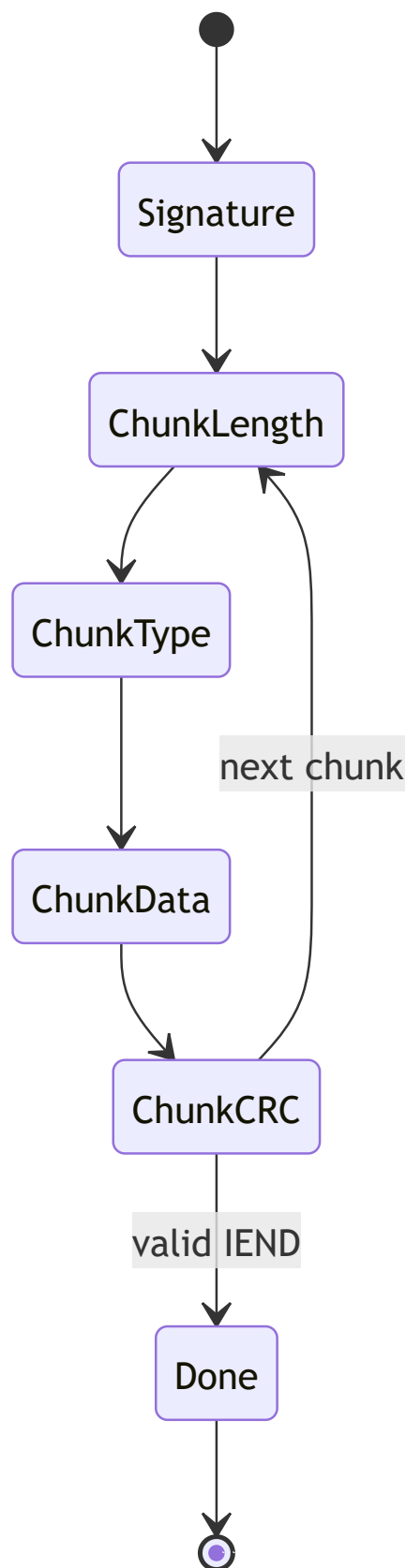
格式	bits_per_pixel	Filter bpp
Grayscale 1-bit	1	1
Indexed 4-bit	4	1
RGB8	24	3
RGBA8	32	4
RGB16	48	6

Filter 操作对象是序列化后的 byte，不是抽象的 Sample 或 Pixel。低位深格式中，一个 byte 包含多个像素，但 Sub 的左邻居仍是前一个 byte。

第四篇 完整 Decoder 数据流

16. Parser 状态机

推荐的顶层状态机：



Parser 应维护:

- 是否已见 IHDR、PLTE、IDAT、IEND;
- 当前是否仍位于连续 IDAT 区间;

- Color Type 与后续 Chunk 的兼容性;
- 当前 Chunk 剩余字节;
- CRC 状态;
- 整体错误状态。

对 IDAT，Chunk Parser 只剥离外壳，把 Data byte 送给同一 zlib 输入接口。

17. 流式解码与缓冲

完整 PNG 解码并不要求：

- 缓存整个文件；
- 缓存所有 IDAT；
- 缓存完整解压结果；
- 缓存完整图像。

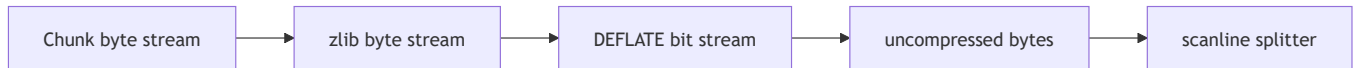
基础流式 Decoder 所需的主要存储：

存储	典型大小	用途
Bit Buffer	数十 bit	DEFLATE 位解析
Huffman Table	数百至数千项	符号解码
LZ77 Window	最大32 KiB	回溯复制
Current Row	row_bytes	当前反滤波
Previous Row	row_bytes	Up/Average/Paeth
Palette	最大256×RGB(A)	Indexed-color

若输出端必须按 Tile 或 Block 接收，则需要额外的行到块重排缓存。这是输出格式需求，不是 PNG 语法本身要求。

18. 边界互不对齐

一个稳健实现应把数据看成连续流，并在不同模块独立维护边界：



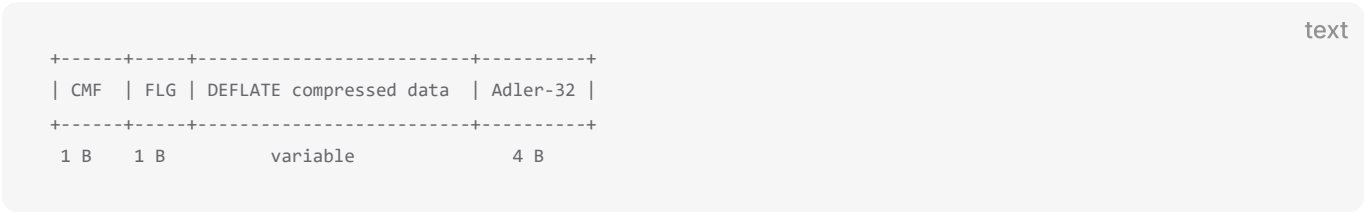
典型场景：

- zlib Header 的两个 byte 可以横跨内部总线传输拍；
- 一个 Dynamic Huffman Header 可以跨 IDAT； 一个 Length/Distance Copy 可以跨 Scanline；
- 一个 DEFLATE Block 可以覆盖多行；
- 一行也可能跨多个 DEFLATE Block。

因此解压器不理解行；行重建器也不应理解 DEFLATE Block。

19. zlib 与 DEFLATE 的关系

zlib 是包装格式，DEFLATE 是内部压缩编码。



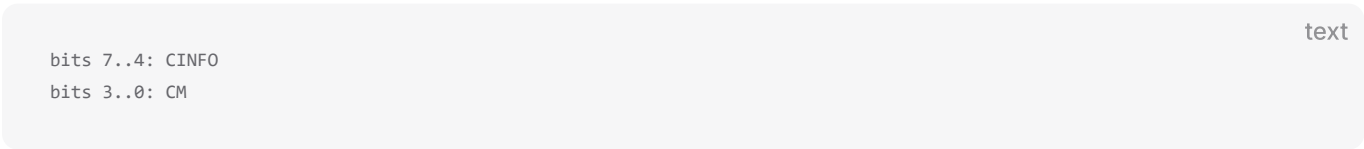
zlib Parser 的职责：

- 1. 解析和检查 CMF、FLG；
- 2. 检查 PNG 不允许的 preset dictionary；
- 3. 把中间位流送给 DEFLATE；
- 4. 对 DEFLATE 输出 byte 计算 Adler-32；
- 5. 与流尾校验值比较。

DEFLATE Decoder 不负责 PNG Chunk CRC，也不负责 Scanline Filter。

20. CMF 与 FLG

CMF:



FLG:

```
bits 7..6: FLEVEL  
bit 5:      FDICT  
bits 4..0: FCHECK
```

text

20.1 CM

PNG Compression Method 0 使用 zlib 的 CM=8，即 DEFLATE。虽然 RFC 1950 为未来压缩方法保留了其他 CM 值，PNG Decoder 不能因此接受任意 CM。

20.2 CINFO

对 CM=8:

```
window_size = 2^(CINFO + 8)
```

text

CINFO 最大为7，对应32768 byte。PNG 使用的窗口不得超过32 KiB；小于32 KiB 的窗口表示也是合法的通用情形，Decoder 的32 KiB Window 能覆盖它。

20.3 FCHECK

两个 Header byte 必须满足：

```
(CMF * 256 + FLG) mod 31 == 0
```

text

20.4 FDICT

PNG 不允许使用 preset dictionary，因此 FDICT 必须为0。若为1，通用 zlib 后面会有 DICTID，但 PNG Decoder 应直接报告非法数据流。

20.5 FLEVEL

FLEVEL 是编码器压缩等级提示，不改变解码算法。Decoder 不应根据它拒绝合法码流。

21. Adler-32

Adler-32 对所有 DEFLATE 解压输出 byte 计算，即覆盖 Filter Type Byte 和 Filtered Scanline Bytes。

初始化：

```
s1 = 1
s2 = 0
MOD = 65521
```

text

对每个输出 byte d：

```
s1 = (s1 + d) mod MOD
s2 = (s2 + s1) mod MOD
```

text

最终：

```
adler = (s2 << 16) | s1
```

text

流中的 Adler-32 以最高有效字节在前存放。高性能实现可批量累加后再取模，但必须限制中间值，避免整数溢出。

CRC 与 Adler-32 的比较：

项目	PNG CRC-32	zlib Adler-32
覆盖对象	每个 Chunk 的 Type+Data	全部未压缩数据
位置	每个 Chunk 尾部	zlib 数据流尾部
主要层次	PNG 封装	zlib 内容
算法	多项式 CRC	两级模和

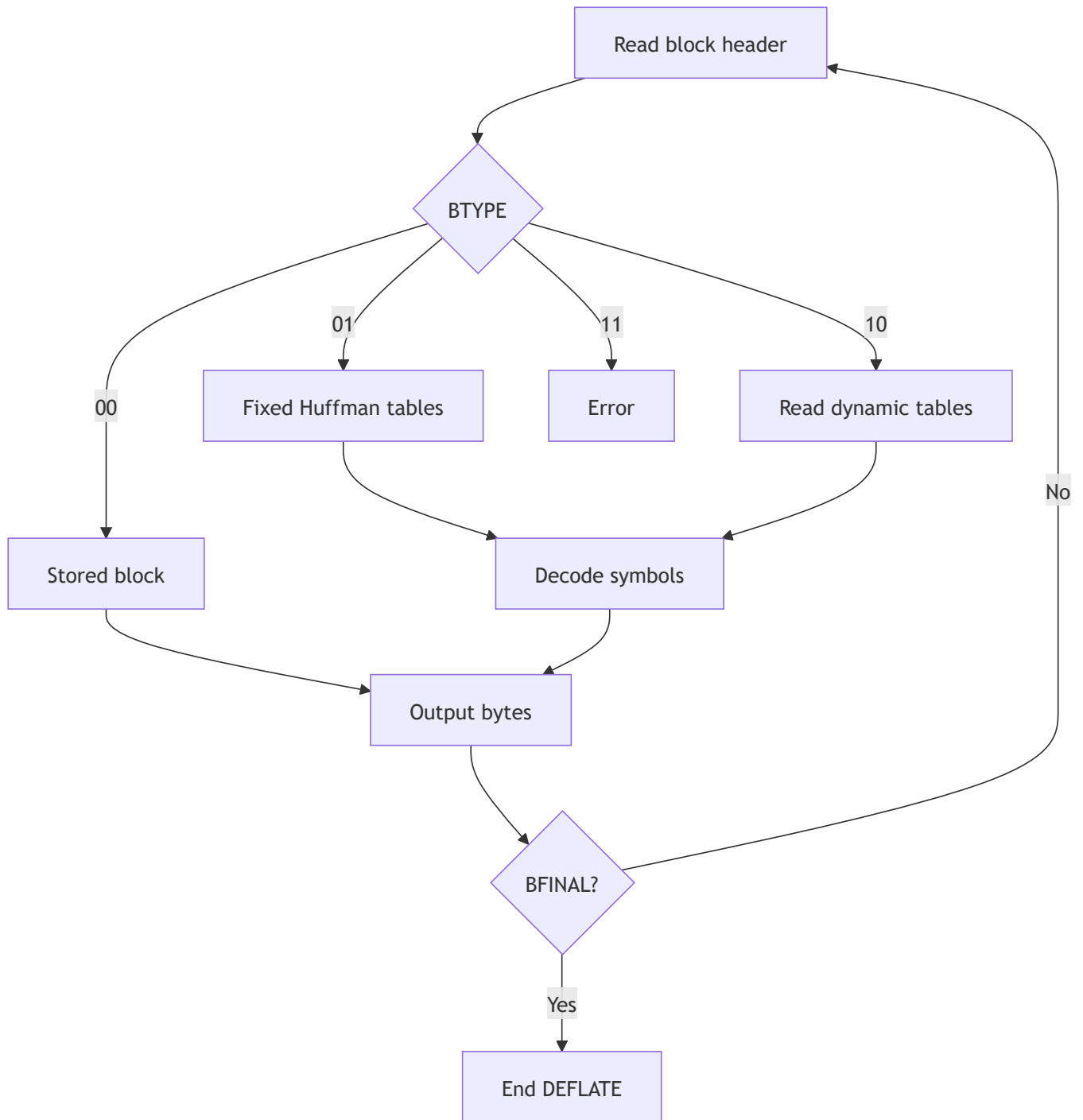
第六篇 DEFLATE 解码

22. DEFLATE 总览

DEFLATE 把输出表示为两类元素：

- Literal: 直接输出一个 byte;
- Match: 从已输出历史中复制 length 个 byte, 源位置距当前输出 distance 个 byte。

Literal/Length 共用一棵 Huffman Tree, Distance 使用另一棵 Tree。



DEFLATE 数据由若干 Block 构成。Block 的 Huffman Tree 独立，但 LZ77 历史窗口不会在 Block 边界清空。

23. 位序

每个 Block 以3个 bit 开始：

text

```
BFINAL: 1 bit
BTYPE:  2 bits
```

DEFLATE 的普通数值字段按最低有效位先进入位流。例如读取 n bit:

text

```
value = bit0 + bit1*2 + bit2*4 + ...
```

Huffman Code 的规范书写方向与其装入 LSB-first Bit Buffer 后的查表方向容易混淆。工程上最稳妥的做法是:

- 1. Canonical 算法生成规范 Code;
- 2. 按 Code Length 将 Code bit reverse;
- 3. 用输入 Bit Buffer 的低位直接查 reverse 后的表。

一句话: 数值字段按 LSB-first 取值; Huffman Code 在规范意义上按最高位到最低位发送, 因此 LSB-first 解码表通常存反转后的 Code。

24. Block Header

BFINAL	含义
0	后面还有 Block
1	当前是最后一个 Block

BTYPE	类型
00	Stored / uncompressed
01	Fixed Huffman
10	Dynamic Huffman
11	Reserved, 错误

BFINAL 只结束当前 DEFLATE 数据集。之后仍要读取 zlib 的 Adler-32。

25. Stored Block

Stored Block 的步骤：

1. 丢弃当前 byte 中剩余未使用 bit，对齐到下一个 byte；
2. 读取 little-endian LEN；
3. 读取 little-endian NLEN；
4. 检查 NLEN 等于 LEN 的16-bit 反码；
5. 原样输出 LEN 个 byte。

```
LEN low | LEN high | NLEN low | NLEN high | raw bytes...
```

text

LEN 最大65535。Stored Block 虽不做压缩，但其输出仍加入 LZ77 History，也参与 Adler-32 和后续 Scanline 解析。

26. Huffman 基础

26.1 Prefix Code

任何有效 Code 都不是另一个 Code 的前缀，所以 Decoder 可以从位流逐 bit 确定符号边界。

26.2 Canonical Huffman

DEFLATE 不发送每个符号的任意 Code，只发送 Code Length。Decoder 按 Canonical 规则恢复 Code：

```
for bits = 1..MAX_BITS:
    code = (code + count[bits-1]) << 1
    next_code[bits] = code

for symbol in increasing symbol order:
    len = length[symbol]
    if len != 0:
        code_for_symbol[symbol] = next_code[len]
        next_code[len]++
```

text

同长度符号按符号值递增分配 Code；短 Code 在数值上排在长 Code 之前。

26.3 合法性检查

用“剩余码空间”可检查是否 oversubscribed：

```
left = 1
for bits = 1..MAX_BITS:
    left = left * 2 - count[bits]
    if left < 0: oversubscribed
```

text

不完整 Tree 的合法性有少数退化例外。实现至少必须拒绝：

- 同一 Code 对应多个符号；
- 码空间超额；
- 解码时落入无符号区域；
- Dynamic Header 无法构造所需的 End-of-block 256。

26.4 Decoder 结构

方法	优点	缺点
逐 bit Tree	简单、面积小	吞吐低、延迟变化大
全长 Lookup	一次查表	表可能很大
一级+二级表	速度与面积平衡	构表和控制更复杂
Canonical 范围比较	存储小	多级比较路径较长

27. Fixed Huffman Block

Literal/Length 固定 Code Length:

Symbol	Code Length
0-143	8
144-255	9
256-279	7
280-287	8

Distance Symbol 0-31 均为5 bit Code，但30、31为保留 Distance Symbol，若实际解码到则错误。

Fixed 表可以在设计时固化，不需要运行时构造。

28. Dynamic Huffman Header

Dynamic Block 首先读取：

字段	bit 数	实际数量
HLIT	5	HLIT + 257
HDIST	5	HDIST + 1
HLEN	4	HLEN + 4

随后读取 Code Length Alphabet 的 Code Length，每项3 bit，顺序固定为：

16, 17, 18, 0, 8, 7, 9, 6, 10, 5, 11, 4, 12, 3, 13, 2, 14, 1, 15

text

未读取到的尾部项 Code Length 为0。用这19个长度先构建 “Code Length Huffman Tree” ，再用它解码最终 Literal/Length Tree 和 Distance Tree 的长度序列。

特殊符号：

Symbol	含义	Extra Bits	重复次数
0-15	直接给出 Code Length	0	1
16	重复前一个非特殊长度	2	3-6
17	重复长度0	3	3-10
18	重复长度0	7	11-138

边界条件：

- Symbol 16 出现在还没有前一个长度时非法；
- 重复后总长度不得超过 HLIT+257 与 HDIST+1 的总和；
- Literal/Length Tree 必须能表示 End-of-block 256；
- 保留的 Literal/Length 286、287 不得作为数据符号使用；
- Distance 30、31 不得使用。

Dynamic Tree 的 “树也被另一棵 Huffman Tree 压缩” 是 DEFLATE 初学者最容易卡住的地方。
解码层次是：

text

```
3-bit code lengths
-> code-length Huffman tree
-> literal/length and distance code lengths
-> two data Huffman trees
-> actual compressed symbols
```

29. Literal、Length 与 Distance

Literal/Length Tree 解码结果:

Symbol	含义
0–255	Literal byte
256	End-of-block
257–285	Length
286–287	Reserved

29.1 Length 表

Code	Base	Extra	Code	Base	Extra
257	3	0	272	31	2
258	4	0	273	35	3
259	5	0	274	43	3
260	6	0	275	51	3
261	7	0	276	59	3
262	8	0	277	67	4
263	9	0	278	83	4
264	10	0	279	99	4
265	11	1	280	115	4
266	13	1	281	131	5
267	15	1	282	163	5
268	17	1	283	195	5
269	19	2	284	227	5
270	23	2	285	258	0
271	27	2			

计算：

```
length = base[code] + read_bits(extra[code])
```

text

29.2 Distance 表

Code	Base	Extra	Code	Base	Extra
0	1	0	15	193	6
1	2	0	16	257	7
2	3	0	17	385	7
3	4	0	18	513	8
4	5	1	19	769	8
5	7	1	20	1025	9
6	9	2	21	1537	9
7	13	2	22	2049	10
8	17	3	23	3073	10
9	25	3	24	4097	11
10	33	4	25	6145	11
11	49	4	26	8193	12
12	65	5	27	12289	12
13	97	5	28	16385	13
14	129	6	29	24577	13

```
distance = base[code] + read_bits(extra[code])
```

text

Distance 不得大于当前有效 History 长度，也不得超过所声明窗口能力。

30. LZ77 Sliding Window

Match Copy 必须按输出顺序逐 byte 进行：

```
repeat length times:
    byte = window[current_position - distance]
    output(byte)
    window[current_position] = byte
    current_position++
```

text

源和目的允许重叠。例如当前已有：

ABC

text

执行 length=8、distance=3：

ABCABCABCAB

text

后面新产生的 byte 会立刻成为后续复制的源。因此普通不支持重叠语义的 memcpy 不是正确模型。环形 Window 中：

```
src = (write_ptr - distance) mod window_size
dst = write_ptr
```

text

每输出一个 byte，src 和 dst 都前进并回绕。

31. 完整 DEFLATE 解码伪代码

text

```
do:
    bfinal = read_bits(1)
    btype = read_bits(2)

    if btype == 0:
        align_to_byte()
        len = read_u16_le()
        nlen = read_u16_le()
        require((len XOR nlen) == 0xFFFF)
        repeat len:
            emit(read_byte())

    else if btype == 1 or btype == 2:
        if btype == 1:
            litlen_tree, dist_tree = fixed_tables()
        else:
            litlen_tree, dist_tree = read_dynamic_tables()

        loop:
            sym = decode(litlen_tree)
            if sym < 256:
                emit(sym)
            else if sym == 256:
                break
            else if 257 <= sym <= 285:
                length = decode_length(sym)
                dsym = decode(dist_tree)
                require(dsym <= 29)
                distance = decode_distance(dsym)
                require(1 <= distance <= valid_history)
                copy_match(distance, length)
            else:
                error()

    else:
        error()

while bfinal == 0
```

每个 emit 同时执行：

1. 写入 LZ77 Window;
2. 更新 Adler-32;
3. 送入 Scanline Splitter;
4. 更新未压缩数据计数。

这样不需要建立完整的中间解压缓冲。

32. DEFLATE 常见错误

- 把 zlib Header 当成 DEFLATE Block Header;
 - 把 IDAT Length 算进压缩数据;
 - BTYPE 读反;
 - Canonical Code 未 bit reverse 就低位查表;
 - Stored Block 未对齐;
 - LEN/NLEN 按大端读取;
 - Dynamic Alphabet 顺序误用 0、1、2.....;
 - Symbol 16 没有前值仍接受;
 - 复制重叠 Match 时一次性拷贝旧数据;
 - Block 结束时清空 Window;
 - 看到 BFINAL 后忘记读取 zlib Adler-32;
 - 把解压输出直接当 RGB 像素。
-

33. Filter 的目标

相邻像素往往相似。若直接压缩 RGB byte，数值本身分布可能很散；若存“当前值减预测值”，大量结果会靠近0，并形成重复模式，更利于 LZ77 和 Huffman。

Filter 是逐行可逆变换。每行可独立选择五种 Filter Type 之一。

定义：

- x: 编码前当前 byte；
- f: Filtered byte；
- a: 当前行左侧 bpp 处已恢复 byte；
- b: 上一行同位置 byte；
- c: 上一行左侧 bpp 处 byte。

缺失的 a、b、c 取0。运算对256取模。

34. 五种 Filter

Type	名称	编码 f	解码 x
0	None	x	f
1	Sub	$x-a$	$f+a$
2	Up	$x-b$	$f+b$
3	Average	$x-\text{floor}((a+b)/2)$	$f+\text{floor}((a+b)/2)$
4	Paeth	$x-\text{Paeth}(a,b,c)$	$f+\text{Paeth}(a,b,c)$

所有结果最终只保留低8 bit。

34.1 None

没有预测依赖，Filtered byte 就是原 byte。它适合已经无明显空间相关性的数据，也常用于最小实现或测试。

34.2 Sub

预测当前 byte 等于同一行前一个像素对应通道：

```
x[i] = f[i] + x[i-bpp]
```

text

前 bpp 个 byte 的 a=0。

34.3 Up

预测当前 byte 等于上一行相同位置：

```
x[i] = f[i] + previous[i]
```

text

第一行或一个 Adam7 Pass 的第一行，b=0。

34.4 Average

```
predictor = floor((a+b)/2)
```

text

必须先用足够宽的整数计算 a+b，避免8-bit 溢出后再除2。

34.5 Paeth

Paeth 用 a、b、c 估计局部二维平面。

```
p = a + b - c
pa = abs(p - a)
pb = abs(p - b)
pc = abs(p - c)

if pa <= pb and pa <= pc: return a
if pb <= pc: return b
return c
```

text

相等时优先级是 a、再 b、再 c。中间值必须使用有符号且足够宽的整数。

35. 逐行反滤波算法

```
filter = input_byte()
require(filter in 0..4)

for i = 0 .. row_bytes-1:
    f = input_byte()
    a = current[i-bpp] if i >= bpp else 0
    b = previous[i] if previous exists else 0
    c = previous[i-bpp] if previous exists and i >= bpp else 0

    predictor = select(filter, a, b, c)
    current[i] = (f + predictor) & 0xFF

consume_or_output(current)
swap(current, previous)
```

text

Filter Type Byte 不参与反滤波公式，也不是 current[0]。

35.1 RGB8 Sub 示例

bpp=3。原始行：

```
10 20 30 | 13 25 29
```

text

Sub 后:

```
10 20 30 | 03 05 FF
```

text

第二个像素 B 分量: $29 - 30 = -1$, 模256后为 FF。解码时 $FF + 30 = 285$, 保留低8 bit 得29。

35.2 Up 示例

上一行:

```
10 20 30
```

text

当前原始行:

```
12 18 35
```

text

Filtered:

```
02 FE 05
```

text

这里 FE 表示 -2 的模256形式, 不代表实际像素很大。

36. 重建依赖

Filter	行内依赖	上一行依赖
None	无	无
Sub	$x[i-bpp]$	无
Up	无	$previous[i]$
Average	$x[i-bpp]$	$previous[i]$
Paeth	$x[i-bpp]$	$previous[i]$ 、 $previous[i-bpp]$

因此“解压后得到类似残差”是合理直觉，但这种残差：

- 没有 DCT/IDCT；
- 没有量化；
- 以 byte 为单位；
- Predictor 由每行第一个 byte 指定；
- Sub/Average/Paeth 具有当前行递推依赖。

37. 从重建行到像素

正确顺序是：

```
DEFLATE output
-> identify row and filter byte
-> unfilter bytes
-> unpack samples
-> palette/tRNS
-> color conversion
-> output pixels
```

text

不能在 Unfilter 前拆解 RGB 通道，因为 Filter bpp 与序列化 byte 位置共同定义左邻域。

38. 七个 Pass

Adam7 依次传输原图的稀疏子集：

Pass	x_start	y_start	x_step	y_step
1	0	0	8	8
2	4	0	8	8
3	0	4	4	8
4	2	0	4	4
5	0	2	2	4
6	1	0	2	2
7	0	1	1	2

以1开始计数的经典位置图：

```
1 6 4 6 2 6 4 6
7 7 7 7 7 7 7 7
5 6 5 6 5 6 5 6
7 7 7 7 7 7 7 7
3 6 4 6 3 6 4 6
7 7 7 7 7 7 7 7
5 6 5 6 5 6 5 6
7 7 7 7 7 7 7 7
```

text

39. Pass 尺寸

若 $\text{width} \leq x_{\text{start}}$, 则 $\text{pass_width}=0$; 否则:

```
pass_width = ceil((width - x_start) / x_step)
            = (width - x_start + x_step - 1) / x_step
```

text

高度同理。Pass 宽或高为0时, 该 Pass 不包含 Scanline, 也没有 Filter Type Byte。

非空 Pass 的:

```
pass_row_bytes = ceil(pass_width * bits_per_pixel / 8)
pass_stream_bytes = pass_height * (1 + pass_row_bytes)
```

text

七个 Pass 的 stream bytes 求和就是预期 DEFLATE 输出长度。

40. Pass 内反滤波

每个 Pass 是独立 Reduced Image:

- 第一行的上一行视为全0;
- Previous Row 不能从上一个 Pass 延续;
- bpp 与原图相同;
- 一行中相邻序列化像素是该 Pass 中相邻采样点, 即原图可能相隔多个坐标。

41. 像素回填

Pass 内坐标 px 、 py 对应原图:

```
x = x_start[pass] + px * x_step[pass]
y = y_start[pass] + py * y_step[pass]
```

text

Decoder 可以:

- 直接随机写最终 Frame Buffer;
- 先生成 Pass 行再由地址生成器 Scatter;
- 软件 Viewer 在每个 Pass 后把已有像素扩展显示, 实现渐进预览。

Adam7 提升渐进体验, 但降低局部连续性, 增加地址生成、写突发组织和缓存复杂度。

第九篇 颜色、Alpha 与输出

42. 各 Color Type 的输出

42.1 Grayscale

低位深灰度通常按满范围缩放到输出位深。例如 n-bit 到8-bit:

```
out = round(sample * 255 / (2^n - 1))
```

text

不要简单左移而导致最大值不能映射到255。

42.2 Truecolor

按 R、G、B 顺序读取。若输出 RGBA，Alpha 补最大值。

42.3 Indexed-color

Sample 是 PLTE Index，不是灰度。先查 RGB，再从 tRNS 查 Alpha；无对应 tRNS 条目的 Alpha 为255。

42.4 带 Alpha 类型

Type 4 为 G、A；Type 6 为 R、G、B、A。PNG Alpha 是 straight alpha。

43. 16-bit 到8-bit

最简单的高字节截断：

```
out8 = sample16 >> 8
```

text

速度快，但不是数学上最准确的满范围舍入。更精确：

```
out8 = round(sample16 * 255 / 65535)
```

text

如果下游支持16-bit，应优先保留原精度。sBIT 描述源数据中的有效精度，可帮助编辑器避免在反复转换中虚构精度。

44. Gamma、颜色和 Alpha

完整 Viewer 的逻辑概念上是：

1. 根据 iCCP、cICP、sRGB、gAMA/cHRM 等确定源颜色空间；
2. 把颜色通道转换到合适的线性或连接空间；
3. 在线性光意义下执行 Alpha 合成；
4. 转换到显示设备空间；
5. 量化到显示格式。

基础硬件 Decoder 常只恢复编码 Sample，把颜色管理交给后级。接口文档必须说明输出是“原始 PNG Sample”还是“已经转换的显示 RGB”，否则不同模块会对 Gamma 和 Alpha 作重复处理。

第十篇 编码器

45. 编码器总流程

```
source pixels
-> choose color type and bit depth
-> optional palette and transparency
-> optional Adam7 extraction
-> serialize each pass into scanlines
-> choose one filter type per scanline
-> DEFLATE all filtered scanlines as one stream
-> add zlib header and Adler-32
-> split bytes into consecutive IDAT chunks
-> add chunks and CRCs
```

text

IDAT 大小是封装选择，不应改变解压结果。编码器可以按便于流式输出或网络传输的尺寸切分。

46. Filter 选择

规范不规定最佳 Filter 选择算法。常见启发式：

1. 对当前行分别计算五种 Filter；
2. 把 Filtered byte 解释为有符号残差；
3. 计算绝对值和；
4. 选择得分最低者。

```
score = sum(abs((int8)filtered_byte))
```

text

它只近似预测 DEFLATE 效果。更高压缩率实现可以在候选行或候选组上实际试压缩，但计算成本更高。

一般经验：

- None：噪声、已经压缩式分布或低开销模式；
 - Sub：水平渐变、宽色带；
 - Up：垂直重复；
 - Average：水平和垂直都相关；
 - Paeth：二维平滑区域和边缘。
-

47. DEFLATE 编码优化

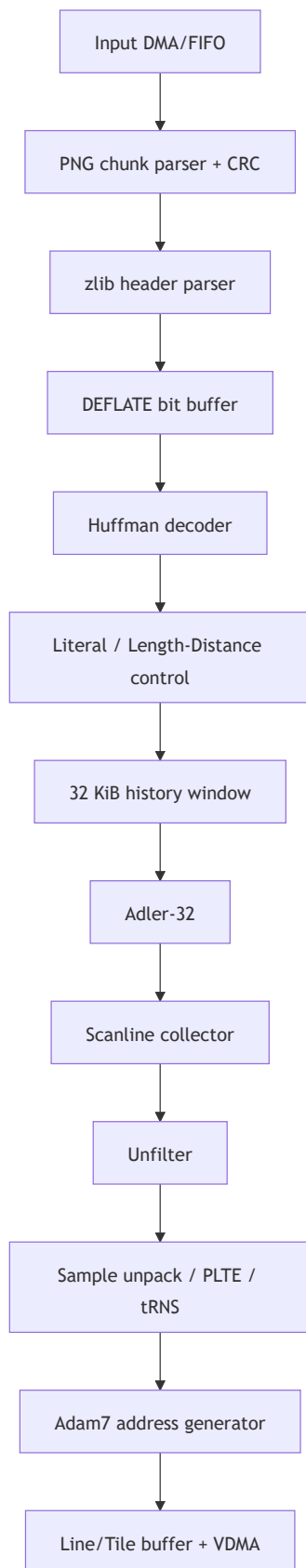
Decoder 是确定的，Encoder 的搜索策略却可复杂很多：

- Hash Table/Hash Chain 查找相同前缀；
- Greedy Match 立即选择当前最长匹配；
- Lazy Match 比较当前位置与下一位置；
- 限制 Search Depth 换速度；
- Block Splitting 在 Stored、Fixed、Dynamic 间估算成本；
- 统计频率并生成限制最大码长的 Huffman Tree；
- 根据 Header 成本判断 Dynamic 是否真正划算。

Compression Level 通常改变搜索深度、Lazy 策略和 Block 决策，不改变格式语义。

第十一篇 硬件架构

48. 顶层模块



各层之间必须支持 valid/ready 或等价流控。下游行缓存或 VDMA 停顿会反压到 DEFLATE；输入端必须能暂停而不丢失 Bit Buffer 状态。

49. 与 JPEG Decoder 的复用

可以复用：

- Host/Register/DMA 框架；
- 输入输出 FIFO；
- 图像尺寸和 stride 管理；
- 像素格式转换中的部分通道逻辑；
- Frame Buffer、VDMA、中断和错误上报。

通常不能直接复用：

- JPEG Marker Parser 作为 PNG Chunk Parser；
- JPEG Huffman 语义作为 DEFLATE Huffman 语义；
- IDCT、反量化路径；
- MCU/Block 调度作为 PNG Scanline 调度；
- JPEG 行输出假设作为 Adam7 Scatter。

JPEG 以频域 Block/MCU 为核心；PNG 以连续 byte stream、LZ77 History 和 Scanline Predictor 为核心。

50. DEFLATE 硬件难点

50.1 位长可变

一次 Symbol 消耗的 bit 数变化，Length/Distance 后还有 Extra Bits。Bit Buffer 需要支持：

- 查看低 N bit；
- 一拍消费可变数量；

- 跨输入 word 补充;
- Byte Alignment;
- Block 类型切换。

50.2 Dynamic Tree 构建

必须先解 Code Length Tree, 再产生两棵数据 Tree。构表阶段没有像素输出, 造成启动延迟。可以用:

- 小型状态机与 RAM;
- Canonical Code 生成单元;
- 一级表加二级表;
- Fixed 表 ROM 与 Dynamic 表 RAM 分离。

50.3 Match Copy 吞吐

distance 小于并行输出宽度时会发生同拍内递归依赖。例如 distance=1 表示重复同一 byte。多 byte/cycle 设计必须:

- 对短周期 Pattern 做展开;
- 或降低该 Match 的输出宽度;
- 或使用迭代旁路网络。

50.4 Window RAM

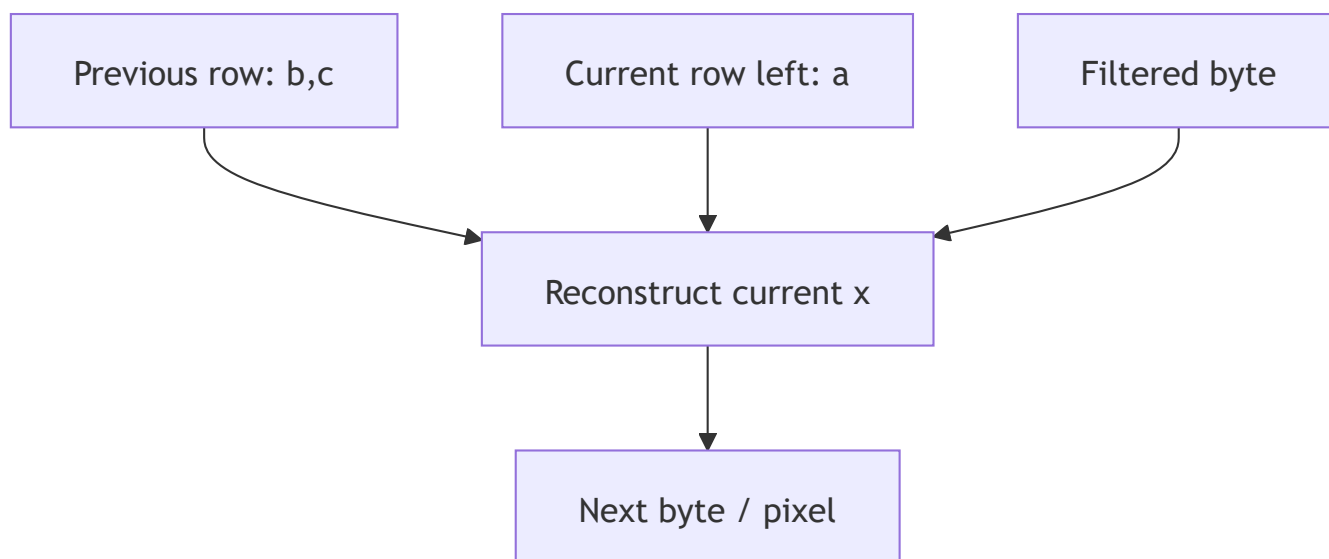
同时存在读源、写目的和可能的多 byte 访问。需考虑:

- Bank 划分;
- Wrap;
- Read-after-write;
- distance 很小时的 Forwarding;

- Block 边界不清空;
- zlib 流开始时有效 History 为0。

51. Scanline 硬件依赖

反滤波依赖图:



None、Up 容易按多 byte 并行; Sub、Average、Paeth 有行内递推。RGB/RGBA 的依赖距离是 bpp, 可在通道对应 lane 之间形成若干链, 但 byte 打包、低位深和总线对齐会使固定 lane 方案复杂化。

Paeth 的组合路径包括加减、绝对值和三级比较, 可能成为高频设计关键路径。可通过:

- 预测值流水;
- 多 lane 错位;
- 小批次前缀展开;
- 降低单周期输出宽度;
- 根据 Filter Type 绕过不用的逻辑。

52. 能否像 WPP 一样多行推进

答案是“可以构造有限的 Wavefront，但不能直接照搬视频编码 WPP”。

限制来自两层：

1. DEFLATE 只按原始序列顺序吐出 byte，后续行数据尚未解压时不能提前重建；
2. Up/Average/Paeth 需要上一行数据，Sub/Average/Paeth 还需要本行左侧数据。

若先缓存多行 Filtered Data，可以让多个 Unfilter Worker 错位推进：

- Row r 的位置 x 依赖 Row $r-1$ 的 x 已完成；
- 对 Average/Paeth，还依赖 Row r 当前 $x-bpp$ 已完成；
- 因而可形成沿 x 方向右移的 Wavefront。

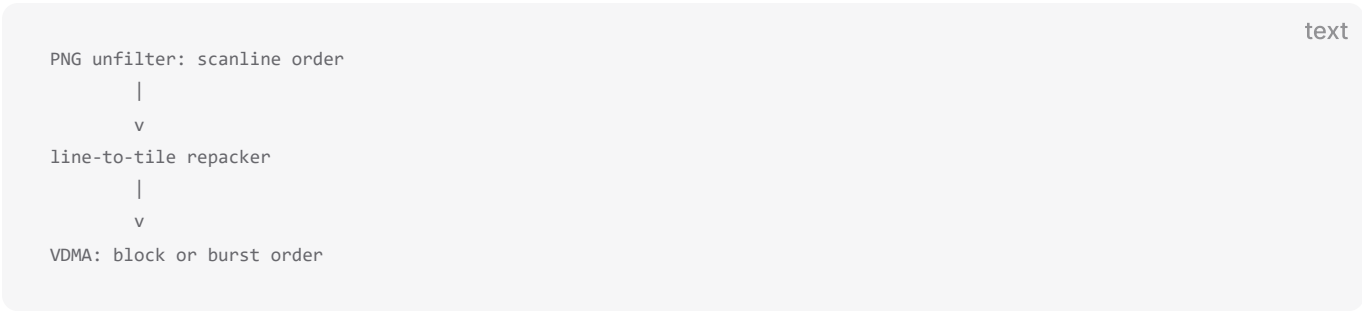
但实际收益取决于：

- DEFLATE 输出是否足以供给多个 Worker；
- 行宽；
- Filter Type 分布；
- SRAM 端口和带宽；
- 输出 VDMA 是否能接受错位多行；
- 为多行数据增加的缓存面积。

缓冲32行并不会自动获得32倍吞吐。若 DEFLATE 平均只产生1 byte/cycle，后级32行并行也无法突破供给瓶颈。

53. 行输出与块输出

PNG 天然按行重建，但系统 VDMA 可能要求 Tile/Block。建议在语义上分两层：



可选方案：

方案	缓存	优点	缺点
VDMA 支持逐行	1-2行	面积最小	需改 VDMA 接口或调度
缓存一个 Tile 高度	tile_h 行	块输出自然	面积随最大宽度增加
缓存完整图像	最大	最简单解耦	通常不可接受
多行环形 Buffer	N行	支持 Wavefront 和 Burst	控制、Bank 和顺序复杂

“32K 行缓存” 与 “32 KiB LZ77 Window” 是完全不同的概念。前者可能极大，后者是 DEFLATE 的固定上限。

54. Buffer 估算

设最大宽度 W、bits_per_pixel=B：

text

```
row_bytes = ceil(W*B/8)
two_row_filter_buffer = 2*row_bytes
N_row_buffer = N*row_bytes
```

例如 32768 像素宽、RGBA8：

text

```
row_bytes = 32768*4 = 131072 bytes
32 rows = 4 MiB
```

如果目标是片上 SRAM，这个规模必须与芯片面积预算对照。工程上往往更合理的是：

- 让 VDMA 接受行或较小 Strip;
- 用外部内存作重排;
- 限制硬件 Profile 的最大宽度;
- 使用少量行缓存配合块化写出。

55. 性能模型

端到端吞吐是最慢阶段的结果:

```
T = min(  
    input byte rate,  
    Huffman symbol rate converted to output bytes,  
    LZ77 copy rate,  
    unfilter byte rate,  
    pixel unpack rate,  
    VDMA acceptance rate  
)
```

text

不能只用 Pixels/Cycle 描述 DEFLATE, 因为一个 Huffman Symbol 可能输出:

- 1 byte Literal;
- 3至258 byte Match;
- 0 byte End-of-block。

建议采集:

- Block 类型计数;
- Literal 和 Match 数;
- 平均 Match Length;
- distance 分布;
- Huffman 表构建周期;
- Window 冲突停顿;
- 各 Filter Type 行数;
- Unfilter 停顿;
- VDMA Backpressure 周期。

56. 分层错误模型

层	典型错误
PNG	Signature、Chunk 顺序、长度、CRC、未知 Critical
IHDR	非法尺寸、Color Type/Bit Depth、Method
zlib	Header mod31、CM、CINFO、FDICT、Adler
DEFLATE	BTYPE=11、Tree 非法、保留符号、越界 Distance、截断
Scanline	非法 Filter Type、输出过多/过少
Pixel	Palette Index 越界、缺失 PLTE
Adam7	Pass 长度不匹配、地址越界

错误上报应保留首个根因，不要让后续级联错误覆盖它。

57. 安全性

解析不可信 PNG 时必须防范：

- widthheightchannels 的整数溢出；
- Chunk Length 诱导的超大分配；
- 极小压缩文件产生极大输出；
- 文本或 ICC Chunk 解压膨胀；
- 截断流导致无限等待；
- 恶意 Dynamic Tree 触发表越界；

- distance 超过有效 History;
- Adam7 坐标计算溢出;
- 过度 CPU 消耗和内存占用。

建议在解码前或解码过程中设置:

- 最大宽高、最大像素数;
- 最大 Chunk 长度;
- 最大元数据解压长度;
- 预期图像数据输出上限;
- 输入与输出超时;
- 所有计数器的饱和或溢出检查。

对静态 PNG, IHDR 已足以计算精确预期 Scanline 输出长度。DEFLATE 输出超过该值应立即报错; 结束时不足也应报错。

58. 验证矩阵

58.1 格式维度

- 五种 Color Type;
- 所有合法 Bit Depth;
- 单 IDAT、多 IDAT、极小 IDAT;
- Ancillary Chunk 前后位置;
- CRC 正确与错误;
- Adam7 0/1。

58.2 DEFLATE 维度

- Stored、Fixed、Dynamic;
- 多 Block;
- 每种 Block 作为最后或非最后;
- 跨 Block LZ77 引用;
- distance=1;
- length=258;
- Window Wrap;
- Dynamic 重复符号16、17、18;
- 不完整、oversubscribed、保留符号和截断流。

58.3 Filter 维度

- 五种 Filter;
- 第一行;
- 行首前 bpp 个 byte;
- width=1;
- 低位深;
- 16-bit;
- Filter Type 每行变化;
- 每个 Adam7 Pass 的第一行。

58.4 参考比对

建议建立：

```
PNG test file
-> reference software decoder
-> RTL/C model under test
-> byte-exact compare at several checkpoints
```

text

检查点包括：

1. IDAT 聚合字节;
2. DEFLATE 输出;
3. 每行 Unfilter 输出;
4. 解包 Sample;
5. 最终 Pixel;
6. CRC、Adler 和错误码。

分层比对能快速判断错误属于 Bitstream、LZ77、Filter 还是像素解释。

第十三篇 贯通实例

59. 最小文件结构

一张最小静态 PNG 至少包含：

```
Signature
IHDR
IDAT
IEND
```

text

逐层解析方法：

1. 识别 Signature;
2. 读取 IHDR，得到尺寸和像素布局;
3. 计算 row_bytes、bpp 和预期解压长度;
4. 对 IDAT 计算 Chunk CRC，同时把 Data 送入 zlib;
5. 检查 CMF/FLG;
6. 解 DEFLATE Block;
7. 对输出更新 Adler;
8. 以 Filter Type+row_bytes 切行;
9. Unfilter;
0. 解包像素;
11. 比较 zlib 尾部 Adler;
2. 验证 IEND。

60. 一个 Filter 算例

假设 RGB8，宽2，上一行和当前 Filtered Data 为：

text

```
previous = [10, 20, 30, 40, 50, 60]
filter   = Paeth
filtered = [ 2,  1, 255,  5,  3,  2]
bpp      = 3
```

前三个 byte 没有左邻居：

- $i=0$: $a=0, b=10, c=0$, Paeth 选 $b=10$, $x=2+10=12$;
- $i=1$: $a=0, b=20, c=0$, $x=1+20=21$;
- $i=2$: $a=0, b=30, c=0$, $x=255+30 \bmod 256=29$ 。

第二个像素开始， a 是当前行前三个已恢复值， b 是上一行同位置， c 是上一行前三个位置。必须使用刚恢复的 $current$ ，而不是 Filtered byte 作为 a 。

61. 完整实现骨架

text

```
decode_png(input):
    require(read(8) == PNG_SIGNATURE)

    state = init_png_state()
    z = init_zlib_state()

    while not state.seen_iend:
        length = read_u32_be()
        type = read(4)
        validate_chunk_header(state, type, length)

        crc = crc32_init()
        crc.update(type)

        repeat length:
            d = read_byte()
            crc.update(d)

            if type == IDAT:
                z.push(d)
            else:
                consume_chunk_data(state, type, d)

        require(crc.finish() == read_u32_be())
        finish_chunk(state, type)

    require(z.finished)
    require(state.image_data_exact_length)
    return state.output
```

真实实现通常让 `z.push` 产生零个或多个未压缩 byte；这些 byte 立即进入 Scanline Reconstructor。

附录 A 速查公式

```
bits_per_pixel = channels * bit_depth
row_bytes = ceil(width * bits_per_pixel / 8)
bpp = max(1, ceil(bits_per_pixel / 8))

non_interlaced_uncompressed_size =
    height * (1 + row_bytes)

pass_width =
    width <= x_start ? 0 :
    ceil((width - x_start) / x_step)

pass_height =
    height <= y_start ? 0 :
    ceil((height - y_start) / y_step)
```

text

附录 B 常见误解

1. “每个 IDAT 都是一个 zlib 流。”

错。所有连续 IDAT Data 拼成一个 zlib 流。

2. “IDAT Data 就是 Raw DEFLATE。”

错。PNG Compression Method 0 使用带 CMF/FLG 和 Adler-32 的 zlib 包装。

3. “DEFLATE 解压后就是 RGB。”

错。先得到 Filter Type 与 Filtered Scanline。

4. “Filter 按像素预测。”

不精确。规范算法按序列化 byte 处理，左邻距离为 bpp。

5. “DEFLATE Block 在每行结束。”

错。Block 与 Scanline 无固定对齐关系。

6. “Block 结束要清空 32 KiB Window。”

错。LZ77 可跨 Block 引用。

7. “有 32 行缓存就能 32 行并行。”

错。还受 DEFLATE 串行输出、Filter 依赖、存储端口和 VDMA 限制。

8. “PNG 的 32K 指行缓存。”

错。通常指 DEFLATE 最大 32768-byte History Window。

9. “Alpha 也做 Gamma Correction。”

错。Alpha 是线性覆盖率。

0. “16-bit PNG 是小端。”

错。PNG Sample 和 Chunk 整数按高字节在前；Stored Block 的 LEN/NLEN 是 DEFLATE 内部例外，按 little-endian。

附录 C APNG 概览

较新 PNG 规范把 APNG 纳入主规范。三个主要 Chunk：

- acTL：总帧数和循环次数；
- fcTL：帧区域、延时、Dispose 和 Blend；
- fdAT：非默认帧的压缩数据，带 Sequence Number。

静态图像仍由普通 IDAT 表示。APNG 的每个帧数据序列使用独立 zlib 数据流，不能把不同帧的压缩数据串成静态图像的同一 zlib 流。兼容静态 PNG 的 Decoder 可以忽略动画 Ancillary Chunk，并显示静态图像。

APNG 详细帧合成属于独立扩展主题；静态 PNG 的 Chunk、zlib、DEFLATE、Filter 和像素恢复知识仍是其基础。

附录 D 规范版本关系

文档	状态与作用
PNG Second Edition, 2003	传统实现最常引用版本；与 ISO/IEC 15948:2004 对应
PNG Third Edition, 2025	W3C Recommendation；整合现代颜色/HDR、Exif、APNG等内容
PNG Fourth Edition	当前编辑草案；用于跟踪演进，不应在未核实状态下当作既有 Recommendation
RFC 1950	zlib Wrapper 与 Adler-32
RFC 1951	DEFLATE 位流、Block、Huffman、LZ77

附录 E 参考资料

1. W3C, Portable Network Graphics (PNG) Specification, Third Edition:
<https://www.w3.org/TR/png-3/>
2. W3C, Portable Network Graphics (PNG) Specification, Fourth Edition Editor's Draft:
<https://w3c.github.io/png/>
3. ISO/IEC 15948:2004, Information technology — Computer graphics and image processing — Portable Network Graphics (PNG): Functional specification.
4. RFC 1950, ZLIB Compressed Data Format Specification version 3.3:
<https://www.rfc-editor.org/rfc/rfc1950>
5. RFC 1951, DEFLATE Compressed Data Format Specification version 1.3:
<https://www.rfc-editor.org/rfc/rfc1951>
6. W3C PNG Test Suite and implementation resources, 见 PNG Specification 的 Online resources。

结语

PNG Decoder 的本质不是一个单独算法，而是一条必须严格分层的可逆数据链：

```
Chunk framing  
-> zlib wrapper  
-> DEFLATE entropy and dictionary decoding  
-> scanline prediction reversal  
-> sample and color interpretation
```

text

对软件实现，分层能降低错误定位成本；对硬件实现，分层能明确状态、缓冲、吞吐和反压边界。尤其应避免把 IDAT、DEFLATE Block、Scanline 和输出 Block 四种边界混为一谈。只要先建立正确的层次模型，再逐层验证，PNG 从格式到算法就会变成一套清晰、可实现、可测试的系统。